

Manual Técnico — CI/CD do ContratacaoMallet

Repositório-base: <https://github.com/luismfrk/ContratacaoMallet>

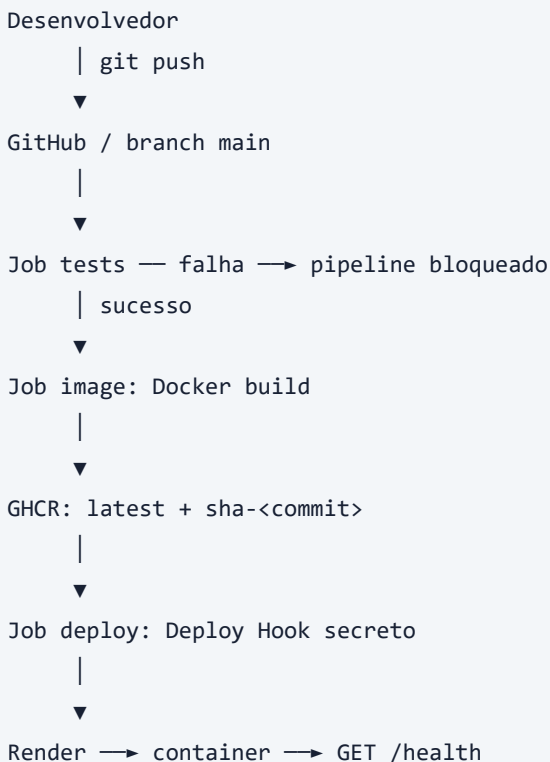
Aplicação: Assistente de Contratações Públicas de Mallet

Stack: Python 3.12, FastAPI, Uvicorn, JavaScript, Docker, GitHub Actions, GHCR e Render

1. Visão geral

O ContratacaoMallet é uma aplicação web para elaboração e gestão de documentos de contratações públicas. O backend em FastAPI serve o frontend JavaScript, autentica usuários, persiste contratações em SQLite ou MySQL e gera documentos DFD, ETP, TR, requisições e relatórios.

O objetivo do pipeline é fazer cada alteração na branch `main` chegar automaticamente à produção somente depois de passar por validação de sintaxe e testes.



Em pull requests somente os testes são executados. Imagem e deploy ocorrem em push para `main` ou execução manual autorizada.

2. Estrutura relevante

```
ContratacaoMallet/
├── .github/workflows/ci-cd.yml # pipeline completo
├── frontend/                  # HTML, CSS, JS, imagens e ícones
├── tests/                     # oito módulos de testes
├── templates/                 # modelos DOC/DOCX
├── server.py                  # FastAPI, rotas e /health
├── database.py                # SQLite/MySQL
├── auth.py                    # usuários, senhas e sessões
├── requirements.txt           # dependências Python
├── Dockerfile                 # imagem de produção
└── .dockerignore              # exclusões do build
```

3. Dockerfile

A construção possui dois estágios. `builder` cria `/opt/venv` e instala as versões permitidas em `requirements.txt`. `runtime` recebe apenas o ambiente virtual e a aplicação.

A base `python:3.12-slim` reduz o tamanho em comparação à imagem completa. O processo roda como usuário `app`, sem privilégios de root. `PYTHONUNBUFFERED=1` envia logs imediatamente e `PYTHONDONTWRITEBYTECODE=1` evita arquivos desnecessários.

Todo o projeto é copiado porque a aplicação depende de `frontend/`, `templates/` e das planilhas-modelo. Testes, documentação, Git, bancos locais, logs e PDFs de exemplo são removidos pelo `.dockerignore`.

O container inicia com:

```
uvicorn server:app --host 0.0.0.0 --port $PORT
```

O `HEALTHCHECK` acessa `/health` por `urllib`, sem instalar curl. Para validar localmente:

```
docker build -t contratacao-mallet .
docker run --rm -p 8000:8000 -e AI_PROVIDER=disabled contratacao-mallet
curl http://localhost:8000/health
```

Resposta esperada: `{"status": "ok"}`.

4. Testes no pipeline

O job `tests` usa Ubuntu e Python 3.12. As etapas são:

1. `actions/checkout@v4` baixa o commit;

2. `actions/setup-python@v5` instala Python e restaura o cache do pip;
3. `pip install -r requirements.txt` instala FastAPI, Uvicorn, python-docx, openpyxl, pdfplumber, PyMySQL e Argon2;
4. `compileall` encontra erros de sintaxe em arquivos Python;
5. `unittest discover` executa todos os arquivos `tests/test_*.py`.

Os testes cobrem autenticação, banco, DFD, DOCX, ETP, ETP de obras, requisições e TR. Durante o CI a IA fica desabilitada e o banco usa arquivos SQLite temporários. Se um teste retorna erro, o job fica vermelho. Como `image` declara `needs: tests`, nenhuma imagem é publicada e nenhum deploy é acionado.

5. Build e publicação no GHCR

O job `image` roda somente depois dos testes e não roda em pull requests. O Buildx constrói a imagem usando cache do próprio GitHub Actions.

A autenticação usa:

- registry: `ghcr.io`;
- usuário: `${{ github.actor }}`;
- senha: `${{ secrets.GITHUB_TOKEN }}`.

O GitHub cria `GITHUB_TOKEN` automaticamente para cada execução. O YAML concede somente `contents: read` e `packages: write`. Não é necessário guardar token pessoal.

A imagem recebe duas tags:

- `ghcr.io/luismfrk/contratacaomallet:latest` — versão atual usada no deploy;
- `ghcr.io/luismfrk/contratacaomallet:sha-XXXXXXX` — versão rastreável do commit.

A tag de SHA permite voltar rapidamente a uma versão estável.

6. Deploy no Render

O Render executará a imagem publicada no GHCR. Configuração inicial:

1. execute o workflow uma vez para criar o package no GHCR;
2. em GitHub **Packages** → **Package settings**, torne o package público; alternativamente, forneça credenciais de leitura ao Render;
3. no Render crie um **Web Service** a partir de uma imagem existente;

4. use `ghcr.io/luismfrk/contratacaomallet:latest`;
5. configure o health check como `/health`;
6. configure as variáveis descritas abaixo;
7. faça o primeiro deploy;
8. copie o **Deploy Hook** do serviço.

No GitHub, acesse **Settings** → **Secrets and variables** → **Actions** → **New repository secret** e crie:

```
RENDER_DEPLOY_HOOK_URL=https://api.render.com/deploy/...
```

O job `deploy` verifica se o valor existe e envia um POST com `curl --fail`. A URL nunca fica exposta no repositório. Recomenda-se criar o environment `production` em **Settings** → **Environments**, restringi-lo à `main` e, se necessário, exigir aprovação.

7. Variáveis de produção

Configure diretamente no Render:

```
AI_PROVIDER=disabled
AI_MODEL=
AI_API_KEY=
COOKIE_SECURE=true
DATABASE_URL=mysql+pymysql://usuario:senha@servidor:3306/contratacoes?charset=utf8mb4
```

`PORT` normalmente é fornecida pelo Render e o Dockerfile a respeita. Não envie `.env` ao GitHub.

Para produção, use MySQL gerenciado. O SQLite padrão fica dentro do container e seus dados podem desaparecer quando uma nova instância for criada. O usuário do banco deve ter acesso apenas à base da aplicação, e a senha deve ser exclusiva.

8. Caminho completo de um push

1. o desenvolvedor testa e envia o commit para `main`;
2. o GitHub cria um runner Ubuntu isolado;
3. dependências são instaladas e a suíte é executada;
4. se tudo passar, o Dockerfile gera a imagem;
5. o Actions autentica com token efêmero e envia `latest` e `sha-*` ao GHCR;

6. o job de produção lê o deploy hook dos secrets;
7. o Render baixa `latest`, inicia Uvicorn e valida `/health`;
8. a nova instância passa a receber tráfego.

9. Teste prático

Antes do push:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

Depois do push:

1. abra a aba **Actions** do repositório;
2. confirme os jobs `Testes Python`, `Build e publicação da imagem` e `Deploy no Render` nessa ordem;
3. confirme no GHCR a tag do commit;
4. consulte os eventos e logs do serviço no Render;
5. acesse `https://URL-D0-SERVICO/health`;
6. abra a página principal e verifique autenticação e geração de documentos.

Para demonstrar o bloqueio, faça a alteração defeituosa somente em uma branch e abra pull request. Um teste vermelho impedirá os demais jobs.

10. Dificuldades e decisões

- **Aplicação possui arquivos além do Python:** templates, frontend e planilhas foram mantidos na imagem; somente artefatos não necessários foram ignorados.
- **Persistência no container:** SQLite serve ao desenvolvimento, mas o manual exige `DATABASE_URL` MySQL em produção.
- **Credenciais:** `GITHUB_TOKEN` é efêmero; hook, banco e IA ficam em secrets/variáveis protegidas.
- **Ordem do processo:** `needs` liga testes, imagem e deploy, bloqueando propagação de falhas.
- **Implantação simultânea:** `concurrency` cancela uma execução antiga quando chega commit mais novo para a mesma referência.
- **Rollback:** tags por SHA preservam a ligação entre imagem e commit.

11. Rollback

Se a versão nova apresentar problema, altere temporariamente no Render a referência da imagem de `latest` para uma tag `sha-XXXXXXX` conhecida, faça deploy e confirme `/health`. A correção deve ser preparada em nova branch, passar pelos testes e então seguir novamente pelo pipeline.

12. Checklist de entrega

- ☒ repositório GitHub definido;
- ☒ sistema real analisado;
- ☒ Dockerfile e `.dockerignore`;
- ☒ testes automatizados no pipeline;
- ☒ build e push no GHCR;
- ☒ secrets e autenticação documentados;
- ☒ deploy automático preparado para Render;
- ☒ diagrama do fluxo e dificuldades;
- ☐ enviar os novos arquivos ao GitHub;
- ☐ configurar GHCR/Render e registrar a URL pública;
- ☐ anexar evidência da primeira execução verde.